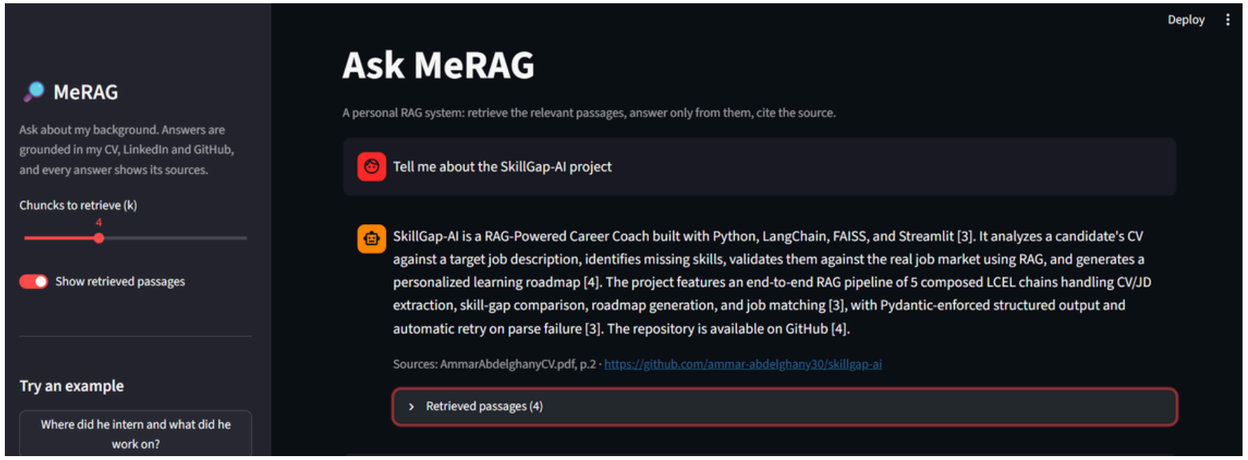
MeRAG

A personal RAG system that answers questions about my background, with source citations

Ammar Abdelghany | Cellula NLP Program | Task 2

The project idea

- Goal: an intelligent knowledge base built from my own data (CV, LinkedIn, GitHub projects)
- Ask a question about me, get an accurate answer grounded in those sources
- Every answer cites its source: file and page, or repo URL
- A plain LLM doesn't know my background and may invent it. RAG makes it answer from evidence



1 Retrieve

Find the passages in my documents most related to the question.

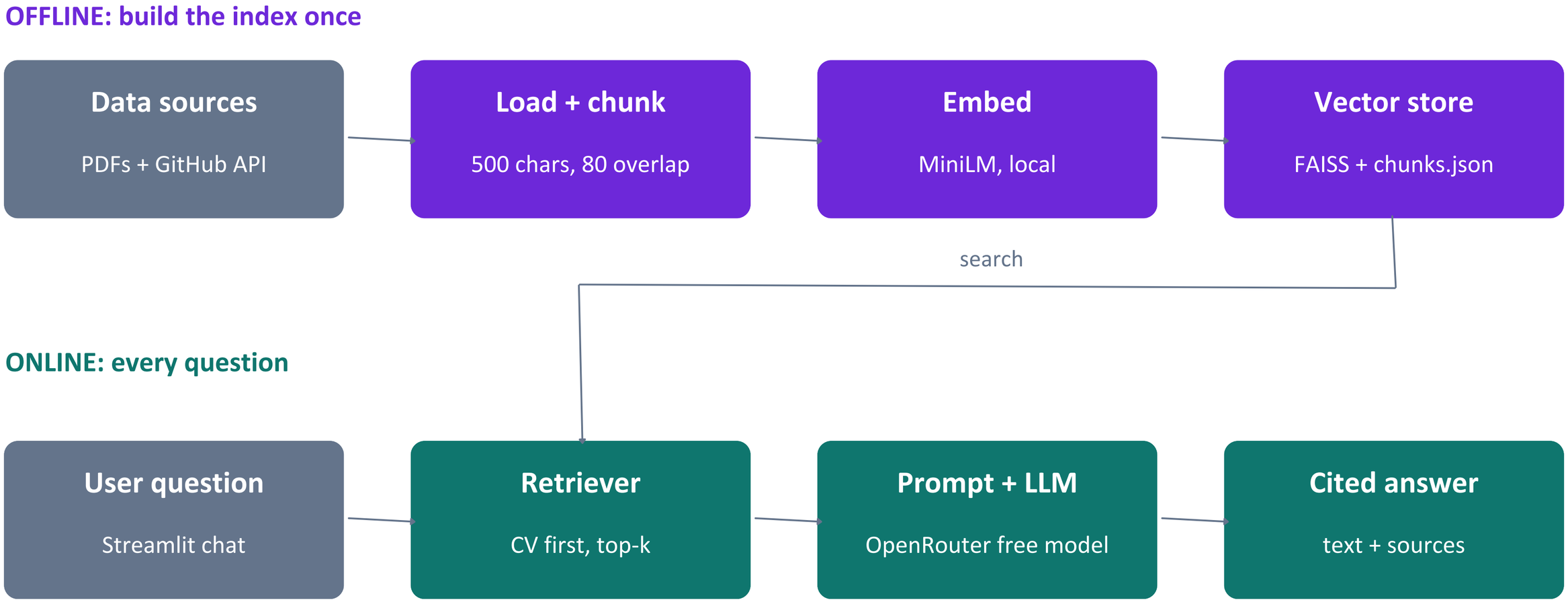
2 Augment

Put those passages in front of the LLM as its only reading material.

3 Generate

The LLM writes the answer from them, and the sources are shown.

System architecture



What each script does

`loader.py`	Reads the CV and LinkedIn PDFs page by page (PyMuPDF) and fetches public GitHub repos and READMEs through the API, as Documents with source info
`chunking.py`	Cleans the text and splits it into 500-character chunks with 80 overlap; keeps the metadata and adds a chunk_id
`vectordb_creation.py`	Embeds chunks with MiniLM, builds the FAISS index, saves chunks.json, and provides search() with an optional metadata filter
`qa.py`	CV-first retrieval, builds the numbered prompt, calls the LLM through OpenRouter, maps [n] citations back to real sources
`app.py`	Streamlit chat interface: answer, sources, retrieved-passages panel, and a k slider for testing

Indexing, chunks.json and metadata

1 Indexing

- 124 chunks of ~500 characters
- MiniLM turns each chunk into 384 numbers
- FAISS stores the vectors in index.faiss
- Similar meaning = nearby numbers (cosine similarity)

2 chunks.json

- FAISS holds only numbers, no text
- chunks.json holds each chunk's text + metadata
- Same order: vector i matches record i
- Both files are written together

3 Metadata

Attached at load time and copied to every chunk by the splitter:

```
{"source": "AmmarAbdelghanyCV.pdf",
 "page": 1, "type": "pdf",
 "chunk_id": 2}
```

Read back by position: used for citations and the CV filter

Retrieval: question > embed > FAISS returns position i > chunks.json[i] > text + metadata > LLM prompt + citation

The LLM and the system prompt

The LLM

- Served through OpenRouter: free models, OpenAI-compatible API
- First model ID is used; the others are automatic fallbacks
- Temperature 0.2: factual, low creativity
- Its only job is to write: it knows nothing about me until the passages arrive

System prompt: 4 rules

1. Use ONLY the numbered context passages
2. If the answer isn't there, say you don't have it
3. If sources disagree, trust the CV
4. Cite passages as [1], [2]; be concise; never invent details

Citations are rebuilt from chunk metadata, so the model cannot invent a source.

The problem: retrieving the right chunks

"Does he know SQL?"

SQL appears in 3 indexed chunks, but none reached the LLM, so it answered "not mentioned".

"How many certificates in his CV?"

Retrieved EEG and SkillGap-AI chunks instead of the CV. Counting also needs the whole list.

"Where did he intern?"

The contact-header chunk outranked the Experience chunk.

Why it happens

- Embeddings match meaning, not exact words or file labels
- Metadata (source = CV) is never seen by the embedding model
- Long, topical GitHub READMEs beat short skill lists

Key insight

The LLM was honest: it only saw the wrong passages. This is a retrieval problem, not a generation problem.

Fix #1: drop the name from the search query

Before: query embedded as-is

`"Where did Ammar intern and what did he work on?"`

Top results were the profile intro and contact header; the Experience chunk ranked below them.

After: name removed

`"Where did intern and what did he work on?"`

The CV Experience chunk ranked #1 and the contact header left the top results.

Why it works

- The name appears in headers, profile intros and contact blocks, so those chunks looked closest to any question that mentions it
- Removing it lets the model focus on the words that carry meaning: intern, work
- Only the embedded query is cleaned; the LLM still receives the original question
- Implementation: a small regex, _strip_name(), applied inside search()

Fix #2: the CV gets priority

1 Reserve CV slots

A search over CV chunks only (type = pdf, file name contains "cv") returns the best 3.

2 Fill the rest

A normal search over all sources adds the best other passages, skipping duplicates, up to 6 total.

3 Prompt rule

The system prompt says: if sources disagree, trust the CV.

```
def retrieve(question, k=6, cv_slots=3):
  cv_hits = search(question, k=cv_slots,
                   where=is_cv)
  merged = list(cv_hits)
  for doc, score in search(question,
                           k + cv_slots):
    if len(merged) >= k: break
    if doc not in merged:
      merged.append((doc, score))
  return merged
```

Settings in qa.py: TOP_K = 6, CV_SLOTS = 3, CV_HINT = "cv"

Why it helps: the CV is ~15 chunks, a much smaller haystack than all 124.


THANK YOU
FOR YOUR
ATTENTION